

CASO DE ESTUDIO

Web Mobile App Gestión Residuos Urbanos

Alumnos:

Pepino, Facundo

Truchet, Mauricio

Docentes:

Ing. Bono, Juan Pablo

Asignatura:

Arquitecturas Móviles

Ciclo Lectivo:

2026

ÍNDICE

1. CONSIGNA 3
2. INTRODUCCIÓN 4
3. REQUERIMIENTOS DEL SISTEMA 5
3.1. Requerimientos funcionales 5
3.2. Requerimientos no funcionales 6
3.3. Alternativas de desarrollo móvil evaluadas 7
4. DISEÑO DE ARQUITECTURA 7
4.1. Estructura general de la solución 7
4.2. Flujo de trabajo y secuencia operativa 9
4.3. Modelo de persistencia y datos en Cloud Firestore 10
4.4. Algoritmo de optimización de rutas de recolección 11
4.5. Arquitectura IoT y protocolo de comunicación para contenedores inteligentes 11
5. PROTOTIPO FUNCIONAL WEB MOBILE 13
5.1. Módulo de autenticación, perfiles y control de acceso 13
5.2. Módulo ciudadano 13
5.3. Panel de control para autoridades municipales 14
5.4. Módulo operativo para personal de cuadrilla 14
5.5. Plataforma de cartografía y geolocalización 15
5.6. Capturas de la interfaz en ejecución 16
6. ESQUEMA DE ESCALABILIDAD Y MONITOREO 17
6.1. Contenerización de servicios con Docker 17
6.2. Orquestación y balanceo con Kubernetes 18
6.3. Autoescalado horizontal de pods (HPA) 19
6.4. Observabilidad y monitoreo con Prometheus y Grafana 19
7. ESTRATEGIA DE PRUEBAS Y CALIDAD 20
8. PROBLEMAS ENCONTRADOS Y SOLUCIONES TÉCNICAS 21
9. ENTREGABLES Y CUMPLIMIENTO DE CONSIGNAS 22
10. INNOVACIÓN Y PARTICIPACIÓN CIUDADANA 23
11. CONCLUSIONES 23
12. BIBLIOGRAFÍA 23

1. CONSIGNA

Caso de Estudio: Web Mobile App Gestión Residuos.

Trabajo Práctico: Desarrollo de Sistema de Gestión de Residuos Vecinales para la Ciudad de San Francisco.

Institución: UTN Facultad Regional San Francisco (UTN FRSFCO). **Carrera:** Ingeniería en Sistemas de Información. **Cátedra:** Arquitecturas Móviles. **Docente:** Ing. Juan Pablo Bono. **Estudiantes:** Facundo Pepino y Mauricio Truchet.

Contexto del enunciado:

La ciudad de San Francisco requiere modernizar la gestión de residuos vecinales. El sistema debe optimizar la recolección, controlar residuos especiales y fomentar la participación ambiental ciudadana.

La solución debe ser eficiente y elástica. Además, coordinará la acción entre vecinos, autoridades municipales y personal de cuadrillas.

Objetivo del Trabajo Práctico:

El objetivo general consiste en diseñar y prototipar una plataforma web móvil y servicios en la nube para la gestión de residuos.

El sistema incorporará interfaces para ciudadanos, autoridades y cuadrillas operativas. Asimismo, aplicará optimización de trayectorias y autoescalado elástico ante demandas crecientes.

Requerimientos establecidos por la cátedra:

- **Aplicación Ciudadana.** Registro de residuos por tipo y volumen. Solicitud de retiro programado para residuos reciclables, voluminosos y peligrosos. Consulta de ecopuntos, horarios de atención y recepción de notificaciones sobre campañas barriales.
- **Aplicación de Autoridades.** Panel municipal con visualización de puntos de recolección en tiempo real. Planificación y asignación de rutas optimizadas para cuadrillas. Monitoreo de telemetría IoT de llenado, alertas de desborde y resolución de incidentes urbanos.
- **Aplicación para Recolectores.** Consulta de hojas de ruta asignadas con confirmación de paradas. Escaneo de códigos QR para autenticar la recolección en contenedores y envío de reportes viales inmediatos.

- **Plataforma de Mapeo y Geolocalización.** Integración con una interfaz de programación de aplicaciones (API) cartográfica para localizar puntos de recolección y generar rutas optimizadas en tiempo real.
- **Notificaciones y Escalabilidad.** Sistema de avisos emergentes (notificaciones push) para vecinos y cuadrillas. Arquitectura escalable basada en contenedores Docker y orquestación con Kubernetes. Herramientas de monitoreo y autoescalado elástico bajo distintas cargas de uso.

Entregables requeridos:

- Diagrama de arquitectura detallado con la interacción entre componentes de frontend, backend, base de datos, cartografía y servicios de escalado.
- Prototipo funcional que permita comprobar el flujo de registro y solicitud ciudadana, junto con el panel municipal y la operatoria de cuadrillas.
- Documentación técnica con la justificación de decisiones arquitectónicas, servicios de contenedores y flujos de trabajo optimizados.
- Esquema formal de escalabilidad y monitoreo con balanceadores de carga, autoescalado elástico y métricas en Prometheus y Grafana.

2. INTRODUCCIÓN

El presente informe desarrolla la solución arquitectónica integral para la gestión de residuos sólidos urbanos en la Ciudad de San Francisco, provincia de Córdoba. El crecimiento demográfico y la dispersión geográfica de los barrios demandan modernizar los métodos tradicionales de recolección mediante una plataforma digital centralizada y reactiva.

La solución articula las necesidades operativas de tres actores fundamentales del ecosistema municipal:

- **Vecinos y comunidad.** Disponen de una aplicación web progresiva para clasificar sus residuos, pedir el retiro gratuito de materiales especiales y consultar los ecopuntos más cercanos a su domicilio.
- **Autoridades municipales.** Acceden a un centro de comando territorial para supervisar el estado de los contenedores, balancear la carga de trabajo entre cuadrillas y resolver reclamos vecinales.
- **Personal de recolección.** Utiliza una aplicación operativa en ruta para recorrer trayectorias optimizadas, validar paradas mediante códigos QR y reportar anomalías viales de forma instantánea.

La arquitectura combina una aplicación web móvil desacoplada con servicios nativos de nube en Google Firebase y un backend contenerizado sobre Kubernetes. Esta combinación

garantiza una disponibilidad continua del 99,9% y un escalado elástico capaz de absorber incrementos bruscos de demanda tras eventos climáticos o campañas comunitarias de descacharrado.

3. REQUERIMIENTOS DEL SISTEMA

3.1. Requerimientos funcionales

Los requerimientos funcionales definen las capacidades operativas del sistema organizadas por cada perfil de usuario.

El siguiente cuadro detalla los requerimientos funcionales aprobados para la plataforma.

Identificador	Rol beneficiario	Descripción funcional	Prioridad
RF-01	Ciudadano	Registrar la generación periódica de residuos indicando categoría y cantidad estimada	Alta
RF-02	Ciudadano	Solicitar la recolección domiciliaria programada para residuos voluminosos y peligrosos	Crítica
RF-03	Ciudadano	Visualizar en un mapa interactivo los ecopuntos, campanas verdes y horarios habilitados	Alta
RF-04	Ciudadano	Recibir notificaciones push sobre campañas ambientales, alertas y avances de sus pedidos	Media
RF-05	Autoridad	Monitorear en tiempo real los puntos de residuos pendientes, en curso y recolectados	Crítica
RF-06	Autoridad	Generar y despachar rutas de recolección optimizadas geográficamente a cada cuadrilla	Crítica
RF-07	Autoridad	Administrar el ciclo de vida de los incidentes urbanos reportados por vecinos y choferes	Alta
RF-08	Autoridad	Emitir notificaciones masivas de concientización ambiental geolocalizadas por barrio	Media
RF-09	Recolector	Consultar la hoja de ruta ordenada de paradas correspondientes al turno laboral	Crítica
RF-10	Recolector	Validar la atención de cada contenedor escaneando su código QR identificador único	Alta
RF-11	Recolector	Confirmar el pesaje real recolectado y registrar observaciones de la parada	Alta
RF-12	Recolector	Reportar incidentes en la vía pública, como contenedores vandalizados o calles obstruidas	Alta

Identificador	Rol beneficiario	Descripción funcional	Prioridad
RF-13	Autoridad	Supervisar la telemetría IoT de llenado y alertas por desborde en contenedores	Crítica
RF-14	Seguridad	Autenticar usuarios mediante Google Identity y validar permisos RBAC por rol	Crítica

Matriz de requerimientos funcionales clasificados por módulo y criticidad operativa.

3.2. Requerimientos no funcionales

Los atributos de calidad condicionan la estructura del software y sustentan las decisiones de infraestructura en la nube.

Las siguientes metas medibles determinan la aceptabilidad técnica del sistema:

- **Disponibilidad del servicio.** La plataforma debe mantener una tasa de actividad del 99,9% durante todo el año, lo que equivale a menos de 8,7 horas de indisponibilidad anual acumulada.
- **Latencia de respuesta.** El percentil 95 de las peticiones de consulta y registro debe responder en un tiempo inferior a los 250 milisegundos bajo condiciones habituales de tráfico.
- **Escalabilidad horizontal elástica.** La infraestructura debe escalar de manera automática desde 2 hasta 10 réplicas cuando el consumo promedio de procesamiento supere el 70%.
- **Sincronización en tiempo real.** Los cambios de estado de una solicitud deben reflejarse en los paneles conectados en menos de un segundo mediante canales reactivos.
- **Soporte ante pérdidas de conectividad.** La aplicación cliente debe almacenar solicitudes e incidentes en la base local del dispositivo y sincronizarlos al recuperar cobertura celular.
- **Seguridad y control de acceso.** Toda comunicación debe transmitirse cifrada por HTTPS, con identidad verificada por Firebase Authentication y permisos por rol exigidos en el servidor.
- **Usabilidad en dispositivos móviles.** La interfaz debe adaptarse a pantallas táctiles desde 360 píxeles de ancho y cumplir con los criterios de accesibilidad AA del estándar web.
- **Eficiencia en el consumo de datos.** El tamaño inicial de transferencia de la aplicación no debe superar los 2 megabytes para asegurar una descarga veloz en redes celulares públicas.

- **Ingesta de telemetría IoT.** El intermediario de mensajería debe procesar y propagar las lecturas en menos de 500 milisegundos garantizando entrega mediante calidad de servicio QoS 1.

3.3. Alternativas de desarrollo móvil evaluadas

La selección de la tecnología cliente evaluó tres paradigmas según su costo, tiempo de puesta en producción y experiencia del usuario.

El siguiente cuadro sintetiza la evaluación de cada enfoque para el municipio de San Francisco.

Enfoque evaluado	Ventajas principales	Desventajas principales	Decisión adoptada
Aplicación Web Progresiva (PWA)	Despliegue inmediato sin comisiones de tiendas, soporte sin conexión y bajo consumo de datos	Acceso limitado a sensores avanzados de hardware	Seleccionada por universalidad y bajo costo de mantenimiento
Aplicación Híbrida (Capacitor/Cordova)	Empaquetado instalable y acceso a periféricos nativos mediante puentes	Mayor complejidad de compilación y demoras en aprobación de tiendas	Descartada por barrera de adopción en trámites ocasionales
Aplicación Nativa (Kotlin/Swift)	Rendimiento óptimo en animaciones complejas y acceso directo a APIs del sistema	Duplicación de código base y costos de desarrollo prohibitivos	Descartada por presupuesto y tiempos de entrega

Comparativa de arquitecturas cliente para la gestión de residuos urbanos.

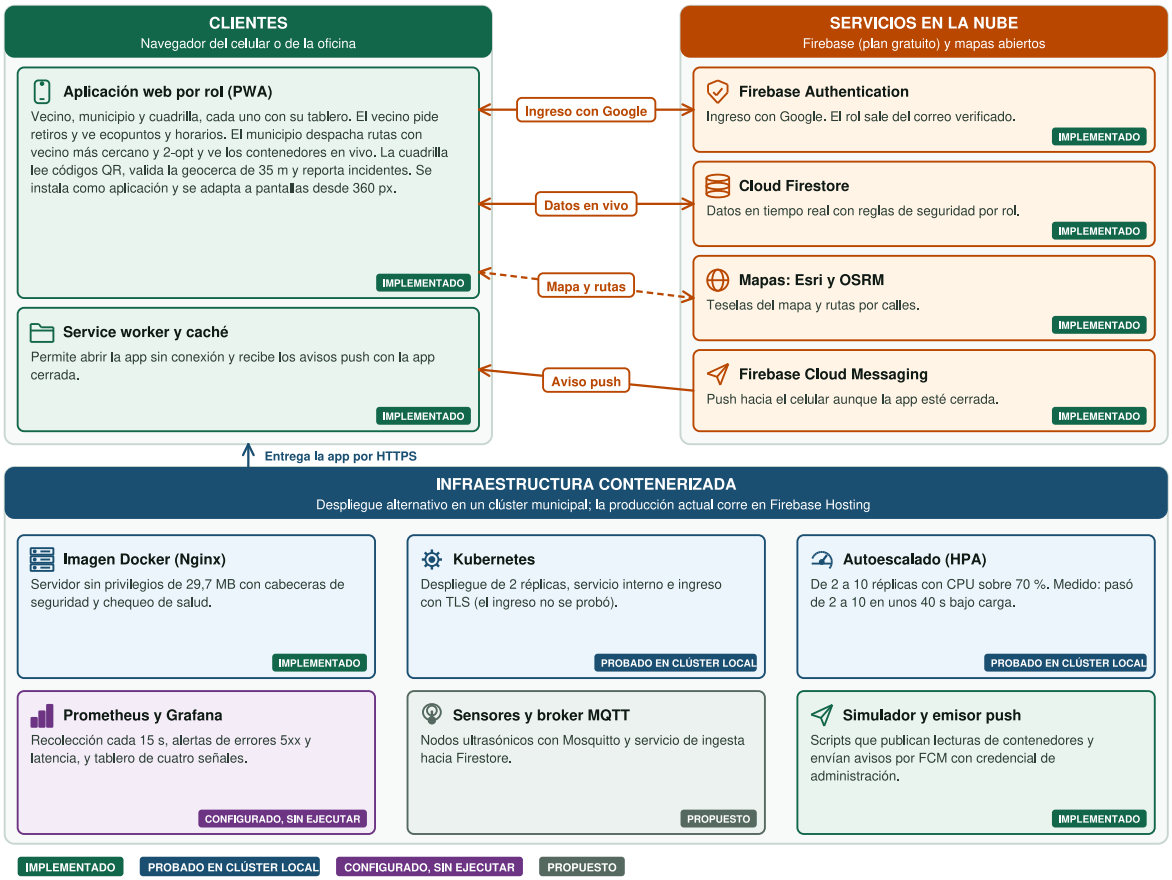
La elección de una aplicación web progresiva responde a que todos los requisitos funcionales se satisfacen desde el navegador moderno. La API de geolocalización resuelve la posición del camión, la cámara web procesa códigos QR y el service worker permite operar sin cobertura celular.

4. DISEÑO DE ARQUITECTURA

4.1. Estructura general de la solución

La arquitectura del sistema adopta un modelo desacoplado basado en tres niveles: clientes móviles enriquecidos, capa de orquestación en contenedores y servicios serverless en la nube. Esta separación permite que cada componente evolucione de forma independiente sin comprometer la estabilidad global del servicio.

El siguiente diagrama presenta los componentes del sistema y marca cuáles están implementados, cuáles se probaron en un clúster local y cuáles son solo una propuesta.



Arquitectura del sistema y estado de avance de cada componente.

La distribución de responsabilidades se organiza en tres bloques principales:

- **Dispositivos clientes.** Alojan la aplicación web responsiva. Un service worker local permite abrir la aplicación sin conexión y recibe los avisos push aunque esté cerrada.
- **Infraestructura contenerizada.** Un controlador de ingreso Nginx aplica terminación TLS, límite de tasa y balanceo entre réplicas. Los microservicios ejecutan sobre Kubernetes y resuelven el algoritmo de optimización de rutas de recolección.
- **Servicios en la nube de Google.** Firebase Authentication identifica a los usuarios con su cuenta de Google. Cloud Firestore centraliza los datos con capacidad reactiva y Firebase Cloud Messaging entrega los avisos push. Los mapas usan teselas de Esri y rutas por calles del servicio OSRM.

4.2. Flujo de trabajo y secuencia operativa

El circuito de recolección coordina la acción simultánea del vecino, la central municipal y el camión recolector en calle.

El diagrama siguiente describe el intercambio secuencial de mensajes desde que se genera un pedido hasta su cierre formal en el centro de transferencia.

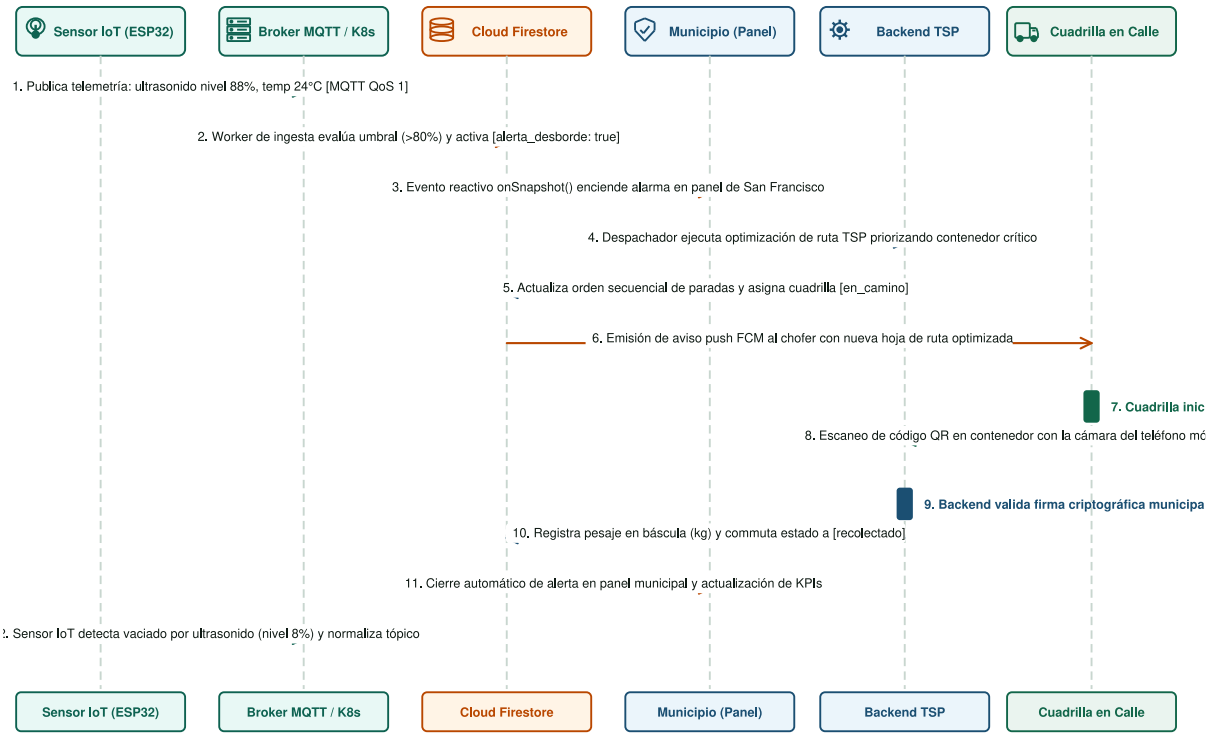


Diagrama de secuencia del circuito de registro, asignación, escaneo y recolección.

El procedimiento operativo comprende las siguientes etapas consecutivas:

- **Generación de la solicitud.** El vecino selecciona la categoría de residuo especial, adjunta una descripción de volumen y confirma su ubicación geográfica en San Francisco. La aplicación persiste el registro en Firestore con estado pendiente.
- **Notificación y optimización municipal.** El panel municipal detecta la solicitud en tiempo real. El despachador agrupa los pedidos pendientes de la zona y ejecuta el algoritmo de ruteo para trazar la ruta más corta.
- **Despacho a cuadrilla.** Al confirmar la ruta, la aplicación actualiza la asignación en Firestore y el móvil asignado recibe en tiempo real un aviso con la nueva parada y el orden del turno.

- **Validación en territorio con código QR.** Al llegar a destino, el operario escanea la placa del contenedor con su cámara o ingresa el identificador. El sistema compara la posición del dispositivo con la del punto y exige estar a menos de 35 metros.
- **Confirmación y cierre de tarea.** El recolector confirma el retiro. Firestore conmuta el estado a recolectado y el vecino recibe de inmediato el aviso de que su solicitud concluyó.

4.3. Modelo de persistencia y datos en Cloud Firestore

La capa de persistencia adopta el motor NoSQL Cloud Firestore debido a su capacidad de sincronización bidireccional en tiempo real y su modelo de seguridad declarativo basado en reglas.

La información se distribuye en cinco colecciones principales estructuradas para evitar lecturas innecesarias:

- **Colección de solicitudes.** Contiene el identificador del vecino, tipo de residuo, subcategoría, volumen estimado, coordenadas geográficas, fecha de registro, estado del trámite y cuadrilla asignada.
- **Colección de ecopuntos.** Registra las campanas verdes y centros limpios instalados en San Francisco, sus coordenadas de latitud y longitud, horarios de atención, materiales aceptados, porcentaje actual de llenado y código QR asociado.
- **Colección de rutas.** Modela la secuencia ordenada de paradas asignadas a cada cuadrilla, la distancia total calculada en kilómetros, el tiempo estimado y el estado de avance del recorrido.
- **Colección de incidentes.** Almacena las anomalías notificadas en vía pública, como contenedores desbordados o microbasurales, incluyendo nivel de gravedad, coordenadas y fotografías probatorias.
- **Colección de notificaciones.** Guarda el historial de avisos emitidos hacia la comunidad o los choferes, registrando título, contenido, fecha de publicación y estado de lectura.

Las reglas de seguridad implementan control de acceso basado en roles (RBAC). Los ciudadanos solo pueden crear y consultar sus propias solicitudes. Las autoridades tienen privilegios de lectura y edición global. Los recolectores únicamente pueden modificar campos operativos delimitados, como el estado de la parada y el pesaje final verificado.

El prototipo sincroniza en la nube las solicitudes, los incidentes, las notificaciones y la telemetría de los contenedores. El catálogo de ecopuntos y las rutas calculadas permanecen en la aplicación.

4.4. Algoritmo de optimización de rutas de recolección

La planificación de recorridos para cuadrillas urbanas resuelve una variante del problema del viajante de comercio (TSP), cuya resolución exacta resulta inviable computacionalmente en tiempo real para múltiples puntos de detención.

Por este motivo, el sistema implementa una combinación de dos algoritmos heurísticos:

- **Heurística del vecino más cercano.** Construye una ruta inicial conectando en cada paso el punto no visitado de menor distancia ortodrómica respecto a la ubicación actual del camión.
- **Optimización por intercambio de aristas (2-opt).** Toma el recorrido inicial e invierte tramos mientras acorten la distancia total, con la base como inicio y cierre. En 1.000 conjuntos aleatorios de paradas dentro de la ciudad, redujo el recorrido en promedio un 5,0% con 5 paradas, un 7,1% con 8 y un 7,4% con 12, sin empeorarlo en ningún caso.

Las distancias entre coordenadas se calculan con la fórmula de Haversine, que contempla la curvatura terrestre. Son distancias en línea recta: incorporar el trazado vial real mediante un servicio de ruteo queda como evolución.

4.5. Arquitectura IoT y protocolo de comunicación para contenedores inteligentes

La modernización del servicio en San Francisco incorpora telemetría en tiempo real sobre los contenedores y ecopuntos distribuidos en la trama urbana. Cada unidad inteligente integra un microcontrolador de bajo consumo con sensores ultrasónicos de distancia y sondas térmicas superficiales.

La selección del protocolo de enlace inalámbrico evaluó los estándares dominantes en sistemas embebidos urbanos.

El siguiente cuadro sintetiza la evaluación comparativa entre las cuatro alternativas tecnológicas consideradas.

Protocolo evaluado	Capa de transporte	Sobrecarga de cabecera	Modelo de comunicación	Decisión arquitectónica
MQTT v5.0 / v3.1.1	TCP / WebSockets	2 bytes fijos	Publicación / Suscripción	Seleccionado por eficiencia, soporte QoS y persistencia
CoAP	UDP	4 bytes fijos	Petición / Respuesta binaria	Descartado por falta de soporte nativo en navegadores
HTTP/2 REST	TCP / TLS	> 500 bytes	Petición / Respuesta síncrona	Descartado por elevado consumo de energía y batería
LoRaWAN directo	Radiofrecuencia sub-GHz	Variable s/SF	Uplink unidireccional	Adoptado en campo físico acoplado vía pasarela MQTT

Comparativa cuantitativa y cualitativa de protocolos de comunicación para telemetría IoT.

La arquitectura adopta el protocolo MQTT por su mínima sobrecarga binaria y su desacoplamiento espacial y temporal mediante intermediarios. Una cabecera fija de apenas dos bytes reduce al mínimo el uso del módem celular y preserva la autonomía energética de los nodos sensores.

El sistema establece tres niveles de operación técnica para garantizar la entrega de telemetría.

- **Calidad de servicio garantizada.** La telemetría regular y las alarmas críticas operan con calidad de servicio QoS 1. Esto asegura la recepción en el intermediario mediante acuse de recibo explícito.
- **Detección de nodo caído.** La conexión registra un mensaje de última voluntad y testamento (LWT). Si un sensor pierde batería o conectividad, el intermediario notifica la anomalía automáticamente.
- **Puente hacia interfaz gráfica.** El intermediario habilita un puerto WebSockets en paralelo al puerto estándar. Esto permite que el centro de comando municipal reciba eventos sin intermediación de servidores adicionales.

La taxonomía jerárquica de tópicos organiza la información según el identificador de cada activo municipal. Los sensores publican mediciones periódicas en sf/contenedores/{id}/telemetria y notifican eventos urgentes en sf/contenedores/{id}/alarmas. A su vez, el centro municipal emite configuraciones y frecuencias de muestreo mediante sf/contenedores/{id}/comandos.

Cada paquete transmitido viaja estructurado en una carga ligera con formato JSON. El mensaje informa el identificador del contenedor, la marca temporal y la distancia en centímetros. También incluye el porcentaje de llenado, la temperatura interna y el nivel de batería restante.

El clúster de Kubernetes aloja el intermediario Eclipse Mosquitto y un servicio de ingesta desacoplado. Dicho componente consume los tópicos mediante patrones comodín, actualiza las marcas de llenado en Firestore e identifica situaciones de saturación.

Cuando un contenedor supera el umbral crítico del 80%, el componente de ingesta dispara una alerta prioritaria. De inmediato, añade la parada a la matriz de optimización de recorridos.

5. PROTOTIPO FUNCIONAL WEB MOBILE

5.1. Módulo de autenticación, perfiles y control de acceso

La plataforma delega la identidad en Firebase Authentication con un único acceso: la cuenta de Google. No hay contraseñas propias ni cuentas de demostración en el código, y cada sesión obtiene un identificador real que Cloud Firestore valida en sus reglas de seguridad.

El control de acceso basado en roles (RBAC) delimita tres niveles. El rol lo decide el servidor: las reglas de Firestore lo otorgan según el correo verificado de la cuenta de Google (dos de autoridad y una de cuadrilla) y lo exigen para operar. Cualquier otra cuenta es de vecino.

- **Perfil Vecino.** Habilita solicitudes de retiro especial y acumulación de puntos ambientales sin acceso a funciones operativas.
- **Perfil Autoridad Municipal.** Otorga supervisión territorial, telemetría de contenedores y despacho algorítmico de rutas.
- **Perfil Personal de Cuadrilla.** Asocia al chofer con una unidad. Autoriza la lectura de hojas de ruta, el escaneo de códigos QR y el pesaje en vía pública.

Cada rol tiene su propio tablero y no hay selector de rol en la navegación: para cambiar de perfil se cierra la sesión y se ingresa con otra cuenta. Las solicitudes se guardan en la nube y llegan en vivo a la autoridad y a la cuadrilla.

5.2. Módulo ciudadano

La aplicación ciudadana se diseñó con un enfoque centrado en la agilidad de uso sobre dispositivos móviles, evitando formularios extensos que desincentiven la participación vecinal.

Las siguientes capacidades componen la interfaz del vecino.

- **Clasificación de residuos.** Permite elegir entre reciclables, voluminosos y peligrosos, con subcategorías para cada uno, como papel y cartón, restos de poda, pilas y baterías.
- **Solicitud de retiro de residuos especiales.** Habilita solicitar la recolección domiciliaria de restos de poda, escombros limpios, pilas, baterías y residuos de aparatos eléctricos y electrónicos (RAEE). El usuario indica la cantidad aproximada y la dirección exacta.
- **Geolocalización de puntos limpios.** Muestra un mapa de San Francisco con los ecopuntos. Al tocar un marcador se consulta la dirección y el nivel de llenado. Debajo, cada punto figura como disponible, casi lleno o lleno.
- **Horarios de recolección.** Una tabla por zona informa los días y horas de recolección domiciliaria, de reciclables y de retiros especiales, y resalta la zona del vecino.
- **Centro de avisos y seguimiento.** Exhibe el historial de pedidos con etiquetas de color según su avance. Recibe en tiempo real las campañas y los cambios de estado de sus solicitudes, y con la aplicación cerrada le llegan como notificación push.

5.3. Panel de control para autoridades municipales

El panel municipal concentra las herramientas de supervisión territorial y despacho requeridas por la Dirección de Servicios Públicos y Medio Ambiente de la ciudad.

El panel ofrece cinco componentes operativos fundamentales.

- **Tablero de métricas.** Presenta indicadores sobre solicitudes pendientes, cuadrillas en calle, incidentes abiertos y puntos limpios operativos.
- **Monitoreo de contenedores en vivo.** Visualiza el llenado, la temperatura y la batería de los contenedores inteligentes, sincronizados en tiempo real desde Cloud Firestore. Permite calibrar los sensores.
- **Despachador inteligente de rutas.** Calcula el recorrido óptimo de las solicitudes marcadas, lo traza por las calles en el mapa y lo asigna a la cuadrilla elegida, que recibe el aviso de inmediato.
- **Consola de incidentes urbanos.** Lista las anomalías reportadas por la cuadrilla, las clasifica por urgencia (alta o media) y permite marcarlas como resueltas.
- **Emisión de campañas.** Difunde avisos que llegan en vivo a vecinos y cuadrillas, con notificación del dispositivo cuando el usuario la autorizó.

5.4. Módulo operativo para personal de cuadrilla

La aplicación del recolector está diseñada para una manipulación rápida en movimiento y con una mano, priorizando la legibilidad en condiciones de luz solar directa.

El flujo del operario abarca las siguientes funciones.

- **Hoja de ruta del turno.** Presenta las paradas del móvil ordenadas por la ruta óptima, con el trazado en el mapa, un resumen del turno y una marca de urgencia en los contenedores casi llenos.
- **Escáner de códigos QR.** Permite escanear la placa de cada contenedor con la cámara o ingresar el identificador a mano. Rechaza los códigos que no pertenecen al catastro municipal.
- **Registro de pesaje y cierre de parada.** Tras vaciar el contenedor o subir el residuo voluminoso, el operario anota el pesaje estimado o verificado y confirma la finalización de la parada.
- **Reporte rápido de incidencias.** Permite registrar con dos toques cualquier anomalía encontrada en calle, como contenedores vandalizados, desbordados o calles anegadas, alertando de inmediato a la central.

Las placas identificadoras de los contenedores y ecopuntos implementan un esquema de código QR estandarizado bajo el siguiente formato de carga útil:

```
sf-residuos://checkin?id=ECO-SF-001
```

La aplicación del recolector procesa la lectura y valida dos condiciones antes de cerrar la parada:

- **Identificador registrado.** El código debe corresponder a un contenedor o a una solicitud existente.
- **Validación territorial por geocerca.** Compara la posición del dispositivo con las coordenadas del punto y exige una proximidad menor a 35 metros. Fuera de ese radio pide una confirmación expresa y deja constancia de que se registró fuera de la geocerca.

5.5. Plataforma de cartografía y geolocalización

La integración cartográfica se resolvió mediante Leaflet y el servicio de teselas Esri World Street Map. Esta combinación brinda un mapa urbano interactivo de alta resolución sin marcas de agua ni restricciones aranceladas.

La decisión de adoptar este proveedor frente a alternativas comerciales o comunitarias responde a criterios técnicos concretos.

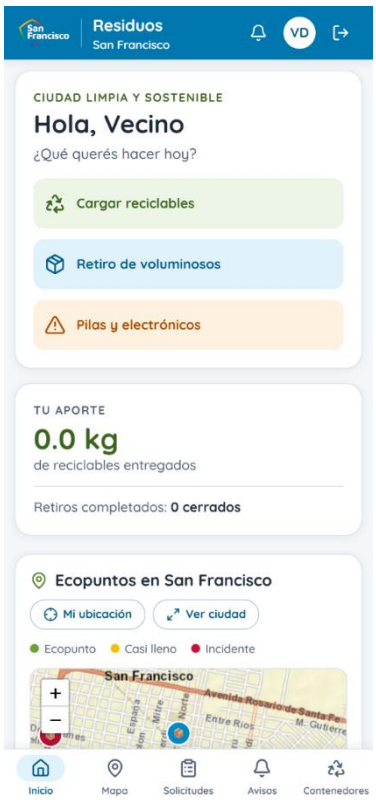
- **Disponibilidad continua sin marcas de agua.** Evita bloqueos por política de uso de servidores comunitarios y marcas comerciales de proveedores arancelados. Asegura cartografía limpia y nítida en toda la ciudad.

- **Soberanía y previsibilidad presupuestaria.** Google Maps impone cobros variables por consumo que comprometen el erario municipal. La solución adoptada garantiza estabilidad operativa y despliegue ilimitado para el municipio.
- **Almacenamiento local y soporte desconectado.** Leaflet permite gestionar teselas urbanas en memoria de almacenamiento local. Esto asegura la visualización fluida del mapa en zonas periféricas sin cobertura celular activa.
- **Rutas por calles.** El recorrido ordenado por el algoritmo se traza con el servicio público OSRM. Si el servicio no responde, el mapa conserva el trazo en línea recta. Es un servicio de demostración: en producción habría que alojar uno propio.

El mapa está centrado en San Francisco (-31,4280 de latitud y -62,0825 de longitud) y muestra los cinco ecopuntos de la ciudad, entre ellos la Plaza Cívica, el Parque Cincuentenario y el Punto Verde de la UTN FRSFCO.

5.6. Capturas de la interfaz en ejecución

Las capturas muestran la aplicación del vecino y de la cuadrilla en el celular, y el panel municipal en pantalla de escritorio.

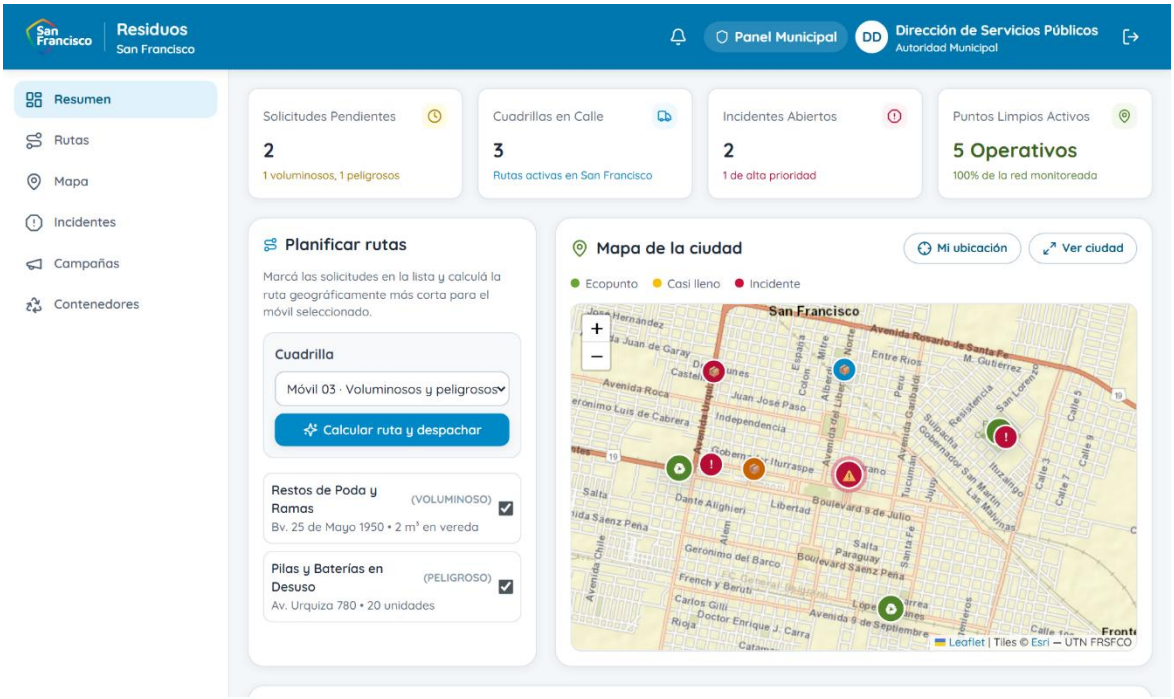


Portal del vecino: registro clasificado y seguimiento de solicitudes.



Aplicación de la cuadrilla: hoja de ruta por calles, escaneo QR y pesaje.

El módulo del vecino promueve la participación con acceso ágil a los puntos limpios, y el de la cuadrilla asegura la trazabilidad del residuo en territorio.



Panel municipal en pantalla de escritorio, con menú lateral.

6. ESQUEMA DE ESCALABILIDAD Y MONITOREO

6.1. Contenerización de servicios con Docker

La aplicación se empaqueta en una imagen mínima basada en Nginx sin privilegios sobre Alpine. Este mecanismo garantiza una imagen liviana, reproducible y segura para su despliegue en cualquier nodo del clúster.

El siguiente cuadro compara la imagen del proyecto con la imagen estándar de Nginx, con valores medidos en el entorno de desarrollo.

Atributo evaluado	Imagen estándar (nginx:latest)	Imagen del proyecto	Beneficio para el sistema
Tamaño en disco	243 MB	29,7 MB	Reduce un 88% el ancho de banda y acelera el inicio de pods
Usuario de ejecución	root (privilegiado)	usuario no privilegiado (UID 101)	Mitiga vulnerabilidades de escape del contenedor
Contenido	Servidor general con configuración por defecto	Únicamente la aplicación y su configuración de seguridad	Disminuye la superficie de ataque
Tiempo hasta responder	1,0 s	0,8 s	Permite respuestas elásticas ante picos de uso

Comparación técnica del empaquetado del contenedor entre desarrollo y producción.

El contenedor incluye comprobaciones automáticas de salud en el puerto 8080 mediante sondas de disponibilidad (`livenessProbe`) y de preparación para recibir tráfico (`readinessProbe`), permitiendo que el orquestador detecte y reinicie cualquier instancia degradada.

6.2. Orquestación y balanceo con Kubernetes

El despliegue de producción se gestiona sobre un clúster de Kubernetes en el espacio de nombres `produccion-residuos`. La configuración desacopla la capa de ruteo perimetral del conjunto de réplicas de cómputo.

Los componentes del clúster operan de la siguiente forma:

- **Controlador de ingreso.** Utiliza Nginx para centralizar el tráfico municipal entrante. Aplica cifrado TLS obligatorio, compresión gzip y un límite estricto de 50 peticiones por segundo por dirección IP para prevenir abusos.
- **Servicio interno (ClusterIP).** Expone un punto de acceso estable y balancea el tráfico entre las réplicas mediante un algoritmo circular (round-robin), aislando la red interna del exterior.
- **Despliegue con estrategia progresiva (RollingUpdate).** Configura un excedente máximo de una instancia y cero indisponibilidad durante las actualizaciones de versión, garantizando que el usuario nunca perciba interrupciones del servicio.
- **Antiafinidad entre pods.** Distribuye las réplicas en distintos nodos físicos del clúster para soportar la caída eventual de una máquina sin que la aplicación se detenga.

6.3. Autoescalado horizontal de pods (HPA)

El autoescalado horizontal elástico (Horizontal Pod Autoscaler) ajusta dinámicamente el número de pods en función del consumo de recursos medido en el clúster.

La política de escalado responde a las siguientes reglas operativas:

- **Umbral de activación.** Cuando el promedio de uso de procesador (CPU) de las réplicas activas supera el 70% de lo solicitado (100 milinúcleos), el controlador HPA incrementa el número de instancias.
- **Rango de réplicas.** El clúster opera con un mínimo de 2 réplicas en horarios nocturnos de baja actividad y escala hasta un máximo de 10 réplicas durante emergencias o eventos comunitarios.
- **Política de escalado hacia arriba acelerada.** Ante un aumento brusco de solicitudes, el sistema duplica las réplicas en 15 segundos para no degradar la latencia de respuesta.
- **Ventana de estabilización hacia abajo.** Cuando la demanda disminuye, el controlador aguarda 300 segundos antes de retirar pods para evitar fluctuaciones continuas de encendido y apagado de contenedores (efecto flapping).

La política se verificó en un clúster local (kind) con la imagen del proyecto. Bajo carga, la CPU superó el 490 % de lo solicitado y el HPA pasó de 2 a 6 réplicas a los 30 segundos y a 10 a los 45. No se esperó la reducción ni se probó el ingreso con TLS. El registro está en `infraestructura/k8s/evidencia_autoescalado.txt`.

6.4. Observabilidad y monitoreo con Prometheus y Grafana

La observabilidad de la arquitectura se sustenta en el ecosistema compuesto por Prometheus para la recolección de series temporales y Grafana para la visualización ejecutiva en tableros en tiempo real.

Las métricas principales y sus criterios de alerta se detallan a continuación:

- **Tasa de solicitudes por segundo (Throughput).** Cuantifica el volumen de tráfico procesado por segundo. Un incremento anómalo activa el autoescalado de réplicas.
- **Latencia en percentiles 95 y 99.** Supervisa el tiempo de resolución percibido por el 95% de los vecinos. Si supera 1,5 segundos durante más de tres minutos, se emite una alerta operativa.
- **Tasa de errores 5xx del servidor.** Mide el porcentaje de respuestas fallidas. Superar el 5% durante dos minutos activa un aviso de guardia para revisar los registros del servicio.

- **Consumo de memoria de pods.** Previene fugas de memoria fijando un límite estricto de 256 megabytes por contenedor antes del reinicio automático.
- **Operaciones en base de datos Firestore.** Monitorea la cantidad de lecturas y escrituras por minuto para asegurar que el sistema se mantenga dentro de los límites presupuestarios del plan de nube.

7. ESTRATEGIA DE PRUEBAS Y CALIDAD

La confiabilidad del sistema se verifica con pruebas automatizadas que se ejecutan contra la copia local, el contenedor Docker y la aplicación publicada.

Las capas de verificación aplicadas antes del pase a producción fueron las siguientes:

- **Flujos de usuario con Playwright.** Recorren cada rol de punta a punta y comprueban el aislamiento de cada tablero.
- **Sincronización entre tres dispositivos.** Vecino, autoridad y cuadrilla operan a la vez y reciben por la nube solicitudes, despachos, campañas e incidentes.
- **Reglas de seguridad de Firestore.** Se prueban contra la base publicada con la cuenta de cada rol y sin sesión.

El siguiente cuadro resume los resultados obtenidos en estas pruebas.

Nivel de prueba	Herramienta utilizada	Cantidad de casos	Resultado obtenido
Flujos de usuario por rol	Playwright Chromium	40 verificaciones	100% exitosas
Sincronización multidispositivo	Playwright, 3 sesiones	18 verificaciones	100% exitosas
Geocerca de 35 metros	Playwright con geolocalización	3 verificaciones	100% exitosas
Reglas de Firestore	API REST con cuentas de cada rol	34 verificaciones	100% exitosas
Push con la aplicación cerrada	Chrome real y Firebase Cloud Messaging	6 verificaciones	Exitosas en la mayoría de las corridas; a veces Chrome automatizado no logra registrar el dispositivo
Optimización de rutas	1.000 conjuntos por tamaño	3 tamaños	Mejora media de 5,0% a 7,4%
Carga sobre el contenedor	Cliente concurrente en Python	50 clientes, 15 s	239 peticiones por segundo, p95 de 261 ms, sin errores del servidor

Nivel de prueba	Herramienta utilizada	Cantidad de casos	Resultado obtenido
Calidad web	Lighthouse	Escritorio y móvil	Accesibilidad 100, prácticas 96, SEO 92, rendimiento 100 en escritorio y de 67 a 93 en móvil
Autoescalado	kind y generador de carga	1 corrida	De 2 a 10 réplicas en 45 s
Ingesta IoT	Simulador de sensores	10 lecturas	10 sincronizadas con Firestore

Resultados medidos de la batería de pruebas sobre el prototipo.

El rendimiento móvil era de 36 puntos. Al compilar los estilos, alojar Leaflet y los íconos en el propio sitio, reducir los íconos de 358 a 34 KB y cargar el lector QR solo al escanear, subió a entre 67 y 93 según la corrida, que varía con la red simulada.

8. PROBLEMAS ENCONTRADOS Y SOLUCIONES TÉCNICAS

Durante las fases de integración del prototipo y despliegue en la infraestructura de nube surgieron dificultades técnicas que exigieron ajustes específicos en la arquitectura.

El siguiente cuadro detalla los principales problemas identificados, su causa raíz y la solución técnica implementada.

Problema identificado	Causa raíz	Solución técnica implementada
Restricciones de acceso y marcas de agua en cartografía	Los servidores de OSM bloqueaban por 403 y CARTO requería clave con marca de agua sobreimpresa	Migración definitiva al servicio de teselas Esri World Street Map de alta resolución y ajuste de CSP
Omisión de cabeceras de seguridad en bloques location	En Nginx la directiva add_header no se hereda automáticamente si un bloque hijo define sus propias cabeceras	Repetición explícita de las cabeceras de seguridad en el bloque de recursos estáticos del servidor
Lectura de códigos QR fuera del radio de atención	El escáner procesaba identificadores de contenedores sin corroborar la proximidad real del vehículo	Validación de la geocerca con un radio máximo de 35 metros y confirmación expresa fuera de él
Riesgo de inyección de código en listados dinámicos	La interpolación directa de valores vecinales en cadenas HTML permitía potenciales vectores de ataque	Incorporación de una función utilitaria de sanitización estricta para neutralizar caracteres reservados

Problema identificado	Causa raíz	Solución técnica implementada
Solicitudes que no llegaban a la nube	El ingreso con cuentas locales no generaba una identidad en Firebase Authentication y las reglas la exigen	Migración a Firebase Authentication con Google
Cuentas de demostración públicas	Las contraseñas de la autoridad y la cuadrilla estaban en el código y cualquiera podía operar como ellas	Roles por correo de Google verificado en las reglas del servidor, cuentas borradas y telemetría reservada a la autoridad
Transmisión de telemetría IoT al navegador sin intermediarios	El protocolo MQTT nativo sobre TCP no es accesible de forma directa desde APIs estándar de JavaScript web	Mosquitto con transporte WebSockets en el puerto 9001 y, en el prototipo, sincronización de la telemetría mediante Cloud Firestore

Registro de problemas técnicos detectados, causas subyacentes y soluciones adoptadas.

9. ENTREGABLES Y CUMPLIMIENTO DE CONSIGNAS

El proyecto comprende un conjunto de artefactos versionados que garantizan la reproducibilidad de la solución y el cumplimiento de las pautas académicas.

Los entregables formales de la solución se estructuran de la siguiente manera:

- **Código fuente de la aplicación móvil.** Interfaz PWA responsiva, cálculo de rutas, lector de códigos QR y soporte sin conexión.
- **Manifiestos de Kubernetes y Dockerfile.** Incluyen la imagen mínima sin privilegios, el controlador de ingreso con TLS, la regla HPA v2 y las políticas de antiafinidad.
- **Configuración de base de datos.** Reglas de seguridad de Firestore por rol e índices compuestos.
- **Infraestructura de monitoreo.** Prometheus con scraping cada 15 segundos, reglas de alerta y tablero de Grafana.
- **Documentación técnica formal.** Informe con diagramas, justificación arquitectónica y resultados medidos.
- **Módulo de telemetría IoT.** Simulador de sensores, servicio de ingesta, intermediario Mosquitto y sus manifiestos.
- **Notificaciones push.** Registro de dispositivos en la aplicación y emisor por Firebase Cloud Messaging.

10. INNOVACIÓN Y PARTICIPACIÓN CIUDADANA

Se proponen dos mejoras para potenciar la adopción ciudadana y la sostenibilidad del servicio:

- **Sistema de incentivos EcoPuntos.** Cada entrega de residuos clasificados acredita puntos canjeables por bonificaciones en tasas municipales. El prototipo ya acumula los puntos del vecino.
- **Mapeo colaborativo de microbasurales.** Los vecinos podrían denunciar focos clandestinos con foto y posición GPS, para una intervención más rápida de las cuadrillas.

11. CONCLUSIONES

El prototipo de gestión de residuos para San Francisco demuestra la viabilidad de modernizar un servicio público esencial con arquitecturas móviles y computación en la nube.

Las principales conclusiones del trabajo son las siguientes:

- **Separación de responsabilidades.** Una aplicación web progresiva con Firebase y servicios en Kubernetes aporta agilidad de desarrollo sin resignar robustez.
- **Optimización de recursos.** El trazado de rutas con 2-opt y la validación en calle permiten reducir los kilómetros recorridos y transparentan el trabajo operativo.
- **Elasticidad verificada en parte.** El autoescalado escaló de 2 a 10 réplicas en un clúster local; el monitoreo con Prometheus y el ingreso con TLS quedan configurados pero sin ejecutar.
- **Impacto en la comunidad.** La geolocalización y los avisos en tiempo real fomentan el compromiso de los vecinos con el cuidado ambiental.

12. BIBLIOGRAFÍA

- Bono, J. P. (2026). Apuntes de Cátedra: Modelos de Arquitecturas basados en Cloud Computing y Escalado de Aplicaciones. UTN Facultad Regional San Francisco.
- Google Cloud. (2026). Cloud Firestore Documentation: Realtime Data and Security Rules Architecture. Google Developers.
- Kubernetes Authors. (2026). Horizontal Pod Autoscaling and Ingress Controller Guidelines. The Linux Foundation.

- Newman, S. (2021). Building Microservices: Designing Fine-Grained Systems (2nd ed.). O'Reilly Media.
- OpenStreetMap Foundation. (2026). OpenStreetMap API and Leaflet Integration Reference. OSMF.
- Prometheus Authors. (2026). *Prometheus Monitoring and Alerting Architecture*. Cloud Native Computing Foundation.
- van Drongelen, M., Dennis, A., Garabedian, R., Gonzalez, A. y Krishnaswamy, A. (2017). *Lean Mobile App Development: Apply Lean startup methodologies to develop successful iOS and Android apps*. Packt Publishing.